MODULE 3 · LECTURE 16

CS 6530 Applied Cryptography

Elliptic-Curve Key Agreement

Module 3 – Public-Key Cryptography

From Elliptic Curves to Cryptography

RECALL

$Q = dG$ d — private scalar Q — public point **Hard reverse problem — ECDLP**

ELLIPTIC-CURVE CRYPTOGRAPHY

KEY ESTABLISHMENT

ECDH • X25519

Establish shared keying material

ENCRYPTION

ECIES • Hybrid encryption

EC-based key establishment + symmetric protection

DIGITAL SIGNATURES

ECDSA • EdDSA / Ed25519

Authenticate signed data

One elliptic-curve foundation supports key establishment, encryption constructions, and digital signatures.

Elliptic Curve Diffie–Hellman (ECDH)

Public parameters: $E,\ G,\ n$ G has order n

ALICE	PUBLIC CHANNEL / EVE	BOB
Private $d_A \in \{1, \ldots, n-1\}$ **Public point** $Q_A = d_A G$ **Alice computes** $S_A = d_A Q_B$ $= d_A d_B G$	$Q_A \rightarrow$ $\leftarrow Q_B$ **EVE SEES** E, G, n, Q_A, Q_B **EVE DOES NOT KNOW** d_A, d_B *Deriving the shared value from public data alone is computationally hard here.*	**Private** $d_B \in \{1, \ldots, n-1\}$ **Public point** $Q_B = d_B G$ **Bob computes** $S_B = d_B Q_A$ $= d_B d_A G$

SHARED EC POINT $S_A = S_B = d_A d_B G$

? THINK LIKE AN ENGINEER

Alice and Bob never transmit the shared EC point. How can both obtain the same point?

INSIGHT

Each combines its private scalar with the other party's public point.

ECDH lets two parties independently derive the same shared EC point without transmitting it.

From Finite-Field DH to ECDH

FINITE-FIELD DH	ECDH
Multiplicative group	Elliptic-curve point group
Generator g	Base point G
Group operation: multiplication	**Group operation: point addition**
Private integer a	Private scalar d_A
$g^a \bmod p$	$d_A G$
$g^{ab} \bmod p$	$d_A d_B G$
DLP	**ECDLP**

Repeated multiplication $\longrightarrow$ Exponentiation g^a

Repeated point addition $\longrightarrow$ Scalar multiplication dG

Same Diffie–Hellman principle + different mathematical group

? THINK LIKE AN ENGINEER

DH and ECDH follow the same key-agreement principle. What changes when we move to the elliptic-curve group?

INSIGHT

The group, group operation, and underlying hard problem change.

ECDH carries the Diffie–Hellman principle into an elliptic-curve group, using point addition and scalar multiplication.

KEY AGREEMENT ≠ ENCRYPTION

ECDH Gives a Common Value — How Do We Protect Data?

? THINK LIKE AN ENGINEER

Can ECDH encrypt a message?

Key agreement and encryption solve different problems.

RSA ENCRYPTION

M ––RSA encryption––▶ C

The message M is an input to the encryption transformation.

ECDH

$d_A, Q_B \rightarrow S_A$

$d_B, Q_A \rightarrow S_B$

No message M is involved in the ECDH computation.

INSIGHT

ECDH establishes a common cryptographic value; it does not transform a message into ciphertext.

WHAT HAPPENS NEXT

ECDH result → **KDF** → Derived keying material → **AEAD** → Protected M

AES-GCM • ChaCha20-Poly1305

ECIES — Elliptic Curve Integrated Encryption Scheme: an EC-based hybrid encryption construction, same broad principle: *ephemeral EC result → KDF → symmetric protection*

ECDH establishes keying material; encryption is performed using symmetric keys derived from the ECDH result.

ECDHE in Practice: Is the Fresh-Key Overhead Worth It?

REUSED / LONGER-LIVED KEY MATERIAL

$d_A \rightarrow Q_A = d_A G$

$d_B \rightarrow Q_B = d_B G$

Key material may be reused across agreements

EPHEMERAL ECDH

For handshake i:

$d_{A,i}^{eph} \rightarrow Q_{A,i}^{eph} = d_{A,i}^{eph} G$

$d_{B,i}^{eph} \rightarrow Q_{B,i}^{eph} = d_{B,i}^{eph} G$

ECDHE = ECDH with fresh ephemeral key pairs

WHAT HAPPENS FOR n HANDSHAKES?

Handshake 1: $(Q_{A,1}^{eph}, Q_{B,1}^{eph})$ Handshake 2: $(Q_{A,2}^{eph}, Q_{B,2}^{eph})$ ⋯ Handshake n: $(Q_{A,n}^{eph}, Q_{B,n}^{eph})$

n handshakes ⇒ $2n$ fresh EC public key shares

$2n$ fresh key shares ≠ $2n$ additional protocol messages

Key shares are normally carried within protocol handshake messages.

COMPUTATION
Fresh scalar multiplication

COMMUNICATION
Fresh key shares transmitted

KEY LIFETIME
Generated, protected, erased

→ *Fresh ephemeral secrets + erasure* **FORWARD SECRECY**

? THINK LIKE AN ENGINEER

Fresh ECDH key pairs add computation and communication to every handshake. Is that overhead justified?

INSIGHT

Yes. The per-handshake cost enables forward secrecy, protecting earlier session keys from later compromise of long-term authentication credentials.

ECDHE trades additional per-handshake computation and fresh key-share transmission for forward secrecy.

ECDHE Still Needs Authentication

MITM RECALL

Alice ⟷ Mallory ⟷ Bob

Mallory substitutes the key shares and forms two separate agreements:

S_{AM} S_{MB}

Key agreement ≠ authentication

Same problem as DH

HOW IS ECDHE AUTHENTICATED?

PATH 1 — CERTIFICATE + SIGNATURE

Ephemeral ECDH exchange + **Certificate / Signature**

⇓ **Authenticated ECDHE**

Long-term signing credentials authenticate and bind the handshake — including the ephemeral exchange — to the intended peer.

PATH 2 — PSK

ECDHE + PSK authentication → Authenticated key establishment

Authentication ↷ *binds and protects the handshake below — it is not a separate sequential step*

KEY AGREEMENT
ECDHE
Establish fresh shared keying material

→

KEY DERIVATION
KDF
Derive usable session keys

→

DATA PROTECTION
AEAD
Protect application data

? THINK LIKE AN ENGINEER

If ECDHE key shares are sent over the network, how can Alice know that Bob really contributed the key share used in the handshake?

INSIGHT

Bind the ephemeral key exchange to authenticated handshake evidence, such as a digital signature/certificate or PSK.

ECDHE establishes fresh keying material; authentication binds the key exchange to the intended peer.

X25519 — A Specific Way to Perform EC Diffie–Hellman

EC Foundations — SHORT WEIERSTRASS FORM

$y^2 = x^3 + ax + b$

⇓

CURVE25519 — MONTGOMERY FORM

Montgomery form: $Bv^2 = u^3 + Au^2 + u$

Curve25519: $v^2 = u^3 + 486662u^2 + u$

over $\mathcal{F}_p$ $(p = 2^{255} - 19)$

Different representations support different implementation techniques.

WHAT IS X25519? **X25519**(k, u)

k: 32-byte scalar input

u: 32-byte Montgomery u-coordinate input

Output: 32-byte result

X25519 specifies scalar processing and Montgomery-ladder scalar multiplication using the u-coordinate.

ALICE AND BOB USE X25519

ALICE

Private a *Public* $A = \mathrm{X25519}(a, 9)$

BOB

Private b *Public* $B = \mathrm{X25519}(b, 9)$

$A \rightleftarrows B$ exchanged over the public channel

Alice: $S_A = \mathrm{X25519}(a, B)$ Bob: $S_B = \mathrm{X25519}(b, A)$ ⇒ $S_A = S_B$

u = 9 is the standard base-point u-coordinate used for generating X25519 public key shares.

X25519 output → KDF / protocol key schedule → session keys

? THINK LIKE AN ENGINEER

We already have ECDH. Is X25519 a different key-agreement principle?

INSIGHT

No. X25519 is a specific, carefully specified way to perform elliptic-curve Diffie–Hellman.

X25519 instantiates the Diffie–Hellman principle using Curve25519 and Montgomery u-coordinate scalar multiplication.

Why X25519?

X25519(k, u)

1 EFFICIENT FIELD ARITHMETIC

$p = 2^{255} - 19$

The field and curve were selected to support efficient finite-field implementation.

2 REGULAR SCALAR MULTIPLICATION

scalar bits → regular ladder steps → result

The Montgomery ladder gives a regular computation structure suitable for constant-time implementation.

3 SIMPLE FIXED-SIZE INPUT / OUTPUT

32-byte k + 32-byte u ⇒ 32-byte output

A simple interface — fixed-size inputs and output — without exposing general EC point operations.

4 MODERN PROTOCOL USE

TLS 1.3 • SSH • modern secure protocols

X25519 is widely standardized and deployed for ephemeral key agreement.

Efficient arithmetic + regular computation + simple input/output
⇓ Practical modern EC key agreement

? THINK LIKE AN ENGINEER

Why standardize a specific EC Diffie–Hellman function instead of asking every application to implement generic EC arithmetic?

INSIGHT

A fixed input/output format reduces implementation choices while supporting efficient, regular scalar multiplication.

X25519 combines efficient arithmetic, regular scalar multiplication and simple fixed-size input/output for modern EC key agreement.

DH • ECDH • X25519 — ENGINEERING COMPARISON

From Finite-Field DH to ECDH to X25519

Engineering View	Finite-Field DH	Generic ECDH	X25519
Group	Finite-field	Elliptic curve	Curve25519 setting
Core operation	Modular exponentiation	EC scalar multiplication	Montgomery ladder
Public value	$g^a \bmod p$	$Q = dG$	32-byte u-coordinate
Hard problem	Finite-field DLP	ECDLP	EC discrete-log hardness
~128-bit target	FFDHE3072	~256-bit EC	X25519
Inputs / representation	Large finite-field values	EC points	Fixed 32-byte values

Approximately 128-bit classical security

FFDHE3072 ↔ **~256-bit EC** ↔ **X25519**

3072-bit modulus — *32-byte public key share*

All three implement the Diffie–Hellman key-agreement principle

? THINK LIKE AN ENGINEER

Why can EC key agreement use much smaller public values than finite-field DH for comparable security?

INSIGHT

Different attack complexity allows much smaller EC parameters for comparable security.

Same DH principle — different groups, parameter sizes and representations.

ECDHE on the Wire: TLS 1.2 → TLS 1.3

TLS 1.2 HANDSHAKE — PACKET LIST (SCHEMATIC)

`ClientHello`

↓

ServerHello • Certificate • ServerKeyExchange • ServerHelloDone

↓

`ClientKeyExchange`

CLIENTKEYEXCHANGE DETAIL

```
EC Diffie-Hellman Client Params
Pubkey Length: 65    Pubkey: 04...
```

CLIENT'S EPHEMERAL EC PUBLIC VALUE:

$$Q_C = d_C G$$

Transmitted on the network

$$04 \parallel X \parallel Y$$

For 256-bit coordinates: 1 + 32 + 32 = 65 bytes

Public EC value is visible ≠ private scalar is revealed

VISIBLE ON THE WIRE: Q_C REMAINS SECRET: d_C

ECDHE → fresh shared keying material Certificate / signature evidence → server authentication

TLS 1.2 — CAPTURE SHOWN
ServerKeyExchange • ClientKeyExchange

⇒

TLS 1.3 — MODERN ORGANIZATION
ClientHello / ServerHello: key_share

ECDHE / X25519 result → HKDF-based TLS 1.3 key schedule → traffic keys

? THINK LIKE AN ENGINEER

The EC public value is visible in this packet capture. Why does this not reveal the shared secret?

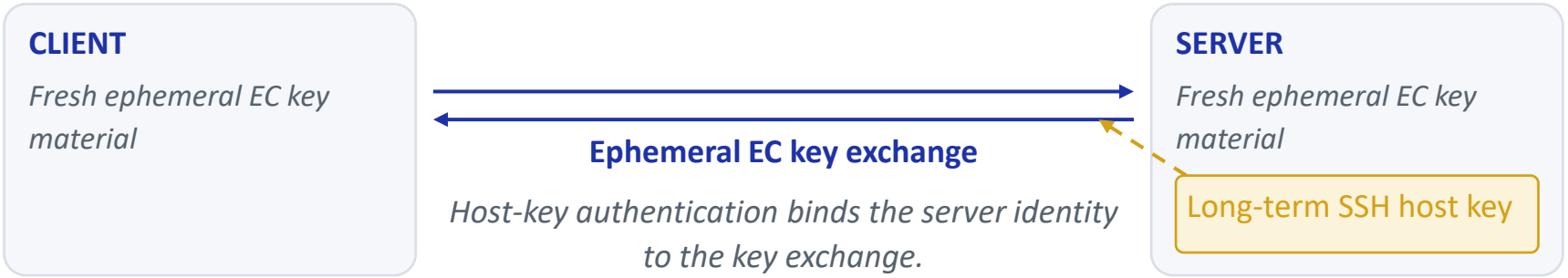
INSIGHT

ECDH is designed for public key shares to be transmitted; deriving the shared result without the private scalar is computationally hard.

TLS carries EC public key shares on the network; private scalars remain secret and the resulting shared value feeds the protocol key schedule.

ECDHE • ANOTHER REAL PROTOCOL

Elliptic-Curve Key Agreement in SSH



Authenticated key establishment
⇓
Derived symmetric keys
⇓
Protected SSH connection

Ephemeral EC key ≠ SSH host key

EPHEMERAL EC KEY
Fresh key-agreement material

SSH HOST KEY
Long-term server authentication credential

EXAMPLE SSH KEY-EXCHANGE METHOD
`curve25519-sha256`

SSH HOST-KEY VERIFICATION
`known_hosts`
The client needs a trusted way to recognize the server's host key.

? **THINK LIKE AN ENGINEER**

If SSH uses ephemeral EC key agreement, why does the server still need a long-term host key?

INSIGHT

The ephemeral exchange establishes fresh shared keying material; the host key authenticates the server and the key exchange.

SSH combines ephemeral key agreement with long-term server authentication, then derives symmetric keys to protect the connection.

Why ECC for Mobile & Resource-Constrained Systems?

APPROXIMATELY 128-BIT CLASSICAL SECURITY: FFDHE3072 vs. roughly 256-bit EC scale

X25519 public key share = 32 bytes *Key-size comparison*

SMARTPHONE | IoT SENSOR / DEVICE | EMBEDDED SYSTEM

Compact EC parameters

1 COMMUNICATION

smaller values → fewer bytes

Useful where bandwidth or communication energy matters.

2 STORAGE / MEMORY

compact keys → less overhead

Can reduce storage and memory requirements associated with cryptographic values.

3 COMPUTATION / ENERGY

efficient EC implementations

Can provide favorable computation and energy trade-offs on constrained platforms.

Bandwidth + Storage / Memory + Computation / Energy

⇓ Cryptographic choice becomes a systems-engineering choice

? THINK LIKE AN ENGINEER

On a constrained device, why does key-exchange size matter even when two schemes provide comparable cryptographic security?

INSIGHT

Communication, storage, computation and energy are engineering costs in addition to cryptographic security.

ECC's compact parameters make elliptic-curve key agreement attractive when communication and device resources matter.

Lecture Takeaways

ELLIPTIC-CURVE CRYPTOGRAPHY

KEY ESTABLISHMENT
ECDH • ECDHE • X25519

ENCRYPTION CONSTRUCTIONS
ECIES • Hybrid encryption

DIGITAL SIGNATURES
ECDSA • EdDSA / Ed25519

One EC foundation supports different cryptographic services.

1
ECDH: SHARED RESULT

$d_A Q_B = d_B Q_A$

Both parties independently derive the same shared EC result.

2
ECDHE: FORWARD SECRECY

fresh pairs→forward secrecy

Fresh ephemeral key pairs protect past sessions.

3
AGREEMENT ≠ AUTHENTICATION

ECDHE + cert/sig or PSK

Authentication binds the key exchange to the intended peer.

4
X25519: PRACTICAL EC

X25519(k, u)

Fixed-size input/output, Montgomery scalar multiplication.

5
RESULT → PROTECTED DATA

result→KDF→keys →AEAD

The shared value feeds key derivation and symmetric protection.

SEEN IN PRACTICE: TLS • SSH • Mobile / constrained systems

Secure key establishment combines a shared cryptographic result with freshness, authentication and key derivation for protected communication.

References

TEXTBOOKS

Jean-Philippe Aumasson — Serious Cryptography

Elliptic-curve cryptography and practical cryptographic engineering.

Christof Paar & Jan Pelzl — Understanding Cryptography

Elliptic curves, ECDLP and elliptic-curve cryptography.

AUTHORITATIVE SPECIFICATIONS

RFC 7748 — Elliptic Curves for Security

X25519 • X448 • Montgomery-curve constructions

RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3

Key shares • ephemeral key agreement • HKDF-based key schedule

RFC 8731 — SSH Key Exchange Method Using Curve25519 and Curve448

Curve25519 / Curve448 key exchange for SSH • curve25519-sha256

RFC 5246 — The Transport Layer Security (TLS) Protocol Version 1.2

Protocol context for the TLS 1.2 Wireshark capture

Textbooks — conceptual foundation • RFCs — algorithm and protocol specifications